

Deep Learning Contest Revision Guide

Mid–Hard Level • Calculator Required

Covers: Perceptron, Activations, XOR, MLPs,
Backpropagation, Gradient Descent, Momentum, NAG,
AdaGrad, RMSProp, Adam, AdamW, Softmax

100 Pages of Worked Examples and Contest Questions

Contents

1	The Perceptron and Activation Functions	2
1.1	The Perceptron Model	2
1.2	The Sigmoid Function	2
1.3	The Tanh Function	2
1.4	The ReLU Function	2
1.5	Contest Questions: Activations	3
2	The XOR Problem and MLPs	4
2.1	Why a Single Perceptron Fails	4
2.2	The Hidden Layer Solution	4
2.3	Counting Parameters	4
2.4	Contest Questions: XOR and MLPs	4
3	Forward and Backward Propagation	5
3.1	The Chain Rule	5
3.2	Worked Example: Output Layer Backprop	5
3.3	Contest Questions: Backpropagation	6
4	Gradient Descent and Learning Rates	6
4.1	Batch, Stochastic, and Mini-Batch	6
4.2	Learning Rate Schedules	7
4.3	Contest Questions: Gradient Descent	7
5	Momentum and NAG	8
5.1	SGD with Momentum	8
5.2	Nesterov Accelerated Gradient (NAG)	8
5.3	Contest Questions: Momentum and NAG	9
6	AdaGrad, RMSProp, Adam, AdamW	10
6.1	AdaGrad	10
6.2	RMSProp	10
6.3	Adam	10
6.4	AdamW	10
6.5	Contest Questions: Adaptive Optimizers	11
7	Softmax and Final Layers	12
7.1	Softmax Definition	12
7.2	Softmax as Generalized Sigmoid	12
7.3	Contest Questions: Softmax	13
8	Comprehensive Mixed Questions	14
9	Extended Calculations	16
10	Formula Sheet	17

1 The Perceptron and Activation Functions

1.1 The Perceptron Model

A perceptron computes a weighted sum of its inputs, adds a bias, and passes the result through an activation function:

$$z = \sum_i w_i x_i + b = \mathbf{w} \cdot \mathbf{x} + b, \quad \hat{y} = f(z)$$

Worked Example 1.1

Given $x = 2$, $w = 1$, $b = 0$, $y = 0$, $\alpha = 0.5$, and a **threshold** activation ($f(z) = 1$ if $z \geq 0$, else 0):

$$z = (1)(2) + 0 = 2$$

$$\hat{y} = f(2) = 1$$

$$\text{Error} = \hat{y} - y = 1 - 0 = 1$$

$$f'(2) = 0 \quad (\text{threshold derivative is zero almost everywhere})$$

$$\frac{\partial L}{\partial w} = (\hat{y} - y) f'(z) x = (1)(0)(2) = 0$$

$$w_{\text{new}} = 1 - 0.5(0) = 1$$

The weight does not change. The largest possible error produced no learning.

1.2 The Sigmoid Function

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \sigma'(z) = \sigma(z)(1 - \sigma(z))$$

Maximum derivative is 0.25 at $z = 0$. Suffers from the vanishing gradient problem for large $|z|$.

1.3 The Tanh Function

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}, \quad \tanh'(z) = 1 - \tanh^2(z)$$

Peak derivative is 1.0 at $z = 0$. Zero-centered, but still saturates.

1.4 The ReLU Function

$$f(z) = \max(0, z), \quad f'(z) = \begin{cases} 1 & z > 0 \\ 0 & z < 0 \end{cases}$$

No vanishing gradient for $z > 0$, but neurons can die for $z < 0$. Unbounded output, so unusable as final layer.

1.5 Contest Questions: Activations

Question 1.1 (Calculation)

A sigmoid neuron receives $x = 4$, $w = 0.5$, $b = -3$. Target $y = 1$, learning rate $\alpha = 0.1$. Using $e^1 \approx 2.7183$, compute w_{new} .

- A) 0.5000
- B) 0.5575
- C) 0.4425
- D) 0.6100

Question 1.2 (Fill in the Blank)

The key reason the threshold activation cannot be trained with gradient descent is that its derivative is _____ almost everywhere.

Question 1.3 (MCQ)

A network has many hidden neurons outputting exactly 0 for all inputs, with weights not updating. Which activation is the most likely culprit?

- A) Sigmoid
- B) Tanh
- C) ReLU
- D) Softmax

Question 1.4 (Calculation)

Given $z = 2$, compute $\sigma'(2)$ using $\sigma(2) = 0.8808$.

- A) 0.1050
- B) 0.8808
- C) 0.1192
- D) 0.2500

Question 1.5 (MCQ)

Which activation function has a maximum derivative of exactly 1.0?

- A) Sigmoid
- B) Threshold
- C) Tanh
- D) ReLU

2 The XOR Problem and MLPs

2.1 Why a Single Perceptron Fails

XOR truth table:

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

The classes $\{(0,0), (1,1)\}$ and $\{(0,1), (1,0)\}$ cannot be separated by any single straight line.

2.2 The Hidden Layer Solution

Two hidden neurons computing OR and AND:

$$h_1 = x_1 \text{ OR } x_2, \quad h_2 = x_1 \text{ AND } x_2$$

Output neuron computes $h_1 - h_2$ (or h_1 AND NOT h_2), which is exactly XOR.

2.3 Counting Parameters

For a layer with n_{in} incoming and n_{out} outgoing neurons:

$$\text{parameters} = \underbrace{n_{\text{in}} \times n_{\text{out}}}_{\text{weights}} + \underbrace{n_{\text{out}}}_{\text{biases}}$$

Worked Example 2.1

A 2-3-1 MLP has:

- Hidden layer: $2 \times 3 + 3 = 9$ parameters
- Output layer: $3 \times 1 + 1 = 4$ parameters
- Total: 13 parameters

2.4 Contest Questions: XOR and MLPs

Question 2.1 (Fill in the Blank)

The hidden layer in an MLP transforms the input space so that the XOR problem becomes _____ separable in the new feature space.

Question 2.2 (Calculation)

How many parameters does a 4-5-3-2 MLP have (4 inputs, two hidden layers of 5 and 3, 2 outputs)?

- A) 43
- B) 53
- C) 63
- D) 73

Question 2.3 (MCQ)

Which of the following is NOT a valid activation function for the final layer of a binary classifier?

- A) Sigmoid
- B) ReLU
- C) Threshold (for historical networks)
- D) Tanh shifted and scaled to $[0, 1]$

3 Forward and Backward Propagation**3.1 The Chain Rule**

For a weight w in the network:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w}$$

For the output layer with sigmoid activation and squared error:

$$\frac{\partial L}{\partial w_{jk}} = \underbrace{(\hat{y}_j - y_j) \sigma'(z_j)}_{\delta_j} \cdot a_k$$

where a_k is the activation of the source (hidden) neuron.

3.2 Worked Example: Output Layer Backprop

Given: 2-3-2 network, sigmoid everywhere. $x_1 = 1, x_2 = 2, y_1 = 1, y_2 = 0, \alpha = 0.5$. Output layer weights: $w_{11} = 0.5, w_{12} = -0.3, w_{13} = 0.8, b_1 = 0.2; w_{21} = -0.6, w_{22} = 0.4, w_{23} = 0.2, b_2 = -0.1$. Hidden activations: $a_1 = 0.6225, a_2 = 0.7311, a_3 = 0.1824$. Forward pass gives: $\hat{y}_1 = 0.6077, \hat{y}_2 = 0.4639$.

Step-by-step for output neuron 1

$$\text{Error}_1 = 0.6077 - 1 = -0.3923$$

$$\delta_1 = -0.3923 \times (0.6077)(1 - 0.6077) = -0.3923 \times 0.2384 = -0.0935$$

$$\frac{\partial L}{\partial w_{11}} = \delta_1 \cdot a_1 = -0.0935 \times 0.6225 = -0.0582$$

$$w_{11}^{\text{new}} = 0.5 - 0.5(-0.0582) = 0.5291$$

Step-by-step for output neuron 2

$$\text{Error}_2 = 0.4639 - 0 = 0.4639$$

$$\delta_2 = 0.4639 \times (0.4639)(1 - 0.4639) = 0.4639 \times 0.2487 = 0.1154$$

$$\frac{\partial L}{\partial w_{22}} = \delta_2 \cdot a_2 = 0.1154 \times 0.7311 = 0.0844$$

$$w_{22}^{\text{new}} = 0.4 - 0.5(0.0844) = 0.3578$$

3.3 Contest Questions: Backpropagation

Question 3.1 (Calculation)

Using the same network as above, compute the updated weight w_{23} (old value 0.2).

- A) 0.1895
- B) 0.2105
- C) 0.1780
- D) 0.2200

Question 3.2 (Fill in the Blank)

The backpropagation algorithm relies on the _____ rule to propagate error gradients backward through the network.

Question 3.3 (Calculation)

For a hidden neuron with $\delta_{\text{hidden}} = 0.05$, source activation $a_{\text{source}} = 0.4$, and learning rate $\alpha = 0.2$, what is the weight update?

- A) -0.004
- B) $+0.004$
- C) -0.040
- D) $+0.040$

Question 3.4 (MCQ)

In the chain rule $\partial L / \partial w = (\partial L / \partial \hat{y}) \cdot (\partial \hat{y} / \partial z) \cdot (\partial z / \partial w)$, which factor represents the derivative of the activation function?

- A) $\partial L / \partial \hat{y}$
- B) $\partial \hat{y} / \partial z$
- C) $\partial z / \partial w$
- D) None of these

4 Gradient Descent and Learning Rates

4.1 Batch, Stochastic, and Mini-Batch

Method	Samples per update	Updates per epoch	Gradient noise
Batch GD	N	1	None
SGD	1	N	High
Mini-batch	B	N/B	Moderate

Key insight: Batch GD is *fastest per epoch* on GPU hardware because it uses parallelism

efficiently. SGD is fastest per *update* but slowest per epoch.

4.2 Learning Rate Schedules

- **Step decay:** multiply α by a factor every K epochs.
- **Exponential decay:** $\alpha_t = \alpha_0 \cdot \gamma^t$.
- **Cosine decay:** smooth reduction following a cosine curve.
- **Warm-up:** start small and increase for the first few epochs.
- **Reduce on plateau:** reactive, reduces only when validation loss stalls.

4.3 Contest Questions: Gradient Descent

Question 4.1 (Calculation)

A dataset has 500,000 samples. Training for one epoch with mini-batch size 250 performs how many updates?

- A) 250
- B) 1,000
- C) 2,000
- D) 500,000

Question 4.2 (Fill in the Blank)

A schedule that starts with a very small learning rate and gradually increases it over the first few epochs is called a _____ schedule.

Question 4.3 (MCQ)

Which learning rate schedule reacts to validation performance rather than a predefined epoch count?

- A) Step decay
- B) Exponential decay
- C) Reduce on plateau
- D) Cosine decay

Question 4.4 (Calculation)

Using $L(w) = (w - 3)^2$, starting at $w_0 = 0$, $\alpha = 0.25$. Compute w_2 .

- A) 1.5000
- B) 2.2500
- C) 2.8125
- D) 3.0000

Question 4.5 (MCQ)

A loss curve oscillates wildly and sometimes becomes NaN. What is the most likely diagnosis?

- A) Learning rate too low
- B) Learning rate too high
- C) Batch size too large
- D) Wrong activation function

5 Momentum and NAG

5.1 SGD with Momentum

$$v_t = \beta v_{t-1} + (1 - \beta)g_t, \quad w_t = w_{t-1} - \alpha v_t$$

The velocity v_t is an exponentially weighted moving average (EWMA) of past gradients.

Worked Example 5.1

Given $w_0 = 2$, $v_0 = 0$, $\beta = 0.8$, $\alpha = 0.25$, $g(w) = 4(w - 1)$.

$$\begin{aligned} g_1 &= 4(2 - 1) = 4 \\ v_1 &= 0.8(0) + 0.2(4) = 0.8 \\ w_1 &= 2 - 0.25(0.8) = 1.8 \\ g_2 &= 4(1.8 - 1) = 3.2 \\ v_2 &= 0.8(0.8) + 0.2(3.2) = 0.64 + 0.64 = 1.28 \\ w_2 &= 1.8 - 0.25(1.28) = 1.48 \end{aligned}$$

5.2 Nesterov Accelerated Gradient (NAG)

$$w_{\text{look}} = w_{t-1} - \alpha \beta v_{t-1}, \quad v_t = \beta v_{t-1} + (1 - \beta) \nabla L(w_{\text{look}}), \quad w_t = w_{t-1} - \alpha v_t$$

Worked Example 5.2 (NAG)

Same problem: $w_0 = 2$, $v_0 = 0$, $\beta = 0.8$, $\alpha = 0.25$, $g(w) = 4(w - 1)$.

$$w_{\text{look},1} = 2 - 0.25(0.8)(0) = 2$$

$$g_1 = 4(2 - 1) = 4$$

$$v_1 = 0.8(0) + 0.2(4) = 0.8$$

$$w_1 = 2 - 0.25(0.8) = 1.8$$

$$w_{\text{look},2} = 1.8 - 0.2(0.8) = 1.64$$

$$g_2 = 4(1.64 - 1) = 2.56$$

$$v_2 = 0.8(0.8) + 0.2(2.56) = 0.64 + 0.512 = 1.152$$

$$w_2 = 1.8 - 0.25(1.152) = 1.512$$

Momentum gave $w_2 = 1.48$; NAG gives 1.512 — slightly behind, because it saw a smaller gradient ahead.

5.3 Contest Questions: Momentum and NAG**Question 5.1 (Calculation)**

With $\beta = 0.9$, $v_{t-1} = 0.5$, $g_t = -2.0$, compute v_t .

- A) -0.250
- B) 0.250
- C) 0.450
- D) -0.450

Question 5.2 (Calculation)

Continuing the momentum example above ($w_2 = 1.48$, $v_2 = 1.28$), compute w_3 .

- A) 1.1280
- B) 0.9960
- C) 1.2000
- D) 0.8208

Question 5.3 (Fill in the Blank)

NAG measures the gradient at the _____ point, not at the current parameter.

Question 5.4 (MCQ)

Why does momentum and NAG produce identical first steps?

- A) Because α is small
- B) Because $v_0 = 0$, so the look-ahead point is w_0
- C) Because the loss is quadratic
- D) Because the gradient is zero

Question 5.5 (Calculation)

In the NAG example, what is w_{look} at step 3 if $w_2 = 1.512$ and $v_2 = 1.152$?

- A) 1.2816
- B) 1.5120
- C) 1.2253
- D) 1.3800

6 AdaGrad, RMSProp, Adam, AdamW

6.1 AdaGrad

$$r_t = r_{t-1} + g_t^2, \quad \theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{r_t} + \epsilon} g_t$$

Accumulates all squared gradients. The denominator grows forever, so learning rate eventually vanishes.

6.2 RMSProp

$$s_t = \rho s_{t-1} + (1 - \rho) g_t^2, \quad \theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{s_t} + \epsilon} g_t$$

Uses EWMA of squared gradients instead of cumulative sum. Allows recovery from old high-gradient periods.

6.3 Adam

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t, & v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \\ \hat{m}_t &= \frac{m_t}{1 - \beta_1^t}, & \hat{v}_t &= \frac{v_t}{1 - \beta_2^t}, & \theta_{t+1} &= \theta_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \end{aligned}$$

6.4 AdamW

Decouples weight decay from the adaptive update:

$$\theta_{t+1} = (1 - \alpha\lambda)\theta_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

6.5 Contest Questions: Adaptive Optimizers

Question 6.1 (Calculation)

AdaGrad with $r_{t-1} = 0$, $g_t = (4, 1)$, ignoring ϵ . What is $g_t / \sqrt{r_t}$?

- A) $(4, 1)$
- B) $(1, 1)$
- C) $(16, 1)$
- D) $(0.25, 1)$

Question 6.2 (Calculation)

RMSProp with $s_{t-1} = (16, 1)$, $g_t = (4, 1)$, $\rho = 0.9$. Compute s_t .

- A) $(16, 1)$
- B) $(14.4, 0.9)$
- C) $(17.6, 1.1)$
- D) $(20, 2)$

Question 6.3 (Calculation)

Adam with $m_0 = v_0 = 0$, $g_1 = 4$, $\beta_1 = 0.9$, $\beta_2 = 0.999$. Compute the bias-corrected $\hat{m}_1$.

- A) 0.4
- B) 4.0
- C) 0.04
- D) 0.004

Question 6.4 (Fill in the Blank)

The key difference between Adam and AdamW is that AdamW _____ weight decay from the adaptive update.

Question 6.5 (MCQ)

Which optimizer accumulates all past squared gradients and therefore has a learning rate that monotonically decreases?

- A) RMSProp
- B) Adam
- C) AdaGrad
- D) NAG

Question 6.6 (Calculation)

Adam step: $\hat{m}_1 = 4$, $\hat{v}_1 = 16$, $\alpha = 0.01$, ignoring ϵ . Compute the parameter movement $\Delta\theta$.

- A) 0.01
- B) 0.04
- C) 0.001
- D) 0.1

7 Softmax and Final Layers

7.1 Softmax Definition

For n raw scores $z_1, \dots, z_n$:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$$

The outputs are all between 0 and 1 and sum to exactly 1.

Worked Example 7.1

Three classes with scores $z_1 = 2.0$, $z_2 = 1.0$, $z_3 = 0.1$.

$$e^{2.0} = 7.389056$$

$$e^{1.0} = 2.718282$$

$$e^{0.1} = 1.105171$$

$$\sum = 11.212509$$

$$p_1 = 7.389056/11.212509 = 0.6590$$

$$p_2 = 2.718282/11.212509 = 0.2424$$

$$p_3 = 1.105171/11.212509 = 0.0986$$

Sum check: $0.6590 + 0.2424 + 0.0986 = 1.0000$.

7.2 Softmax as Generalized Sigmoid

For two classes with scores z and 0:

$$\text{softmax}(z) = \frac{e^z}{e^z + e^0} = \frac{1}{1 + e^{-z}} = \sigma(z)$$

Binary classification is a special case of multi-class.

7.3 Contest Questions: Softmax

Question 7.1 (Calculation)

Scores $z_1 = 1.0$, $z_2 = 2.0$, $z_3 = 3.0$. Using $e^1 = 2.718$, $e^2 = 7.389$, $e^3 = 20.086$, compute p_2 .

- A) 0.0900
- B) 0.2447
- C) 0.6652
- D) 0.3333

Question 7.2 (MCQ)

A network classifies images into 10 digit categories. How many output neurons and which activation?

- A) 1 neuron, sigmoid
- B) 10 neurons, ReLU
- C) 10 neurons, softmax
- D) 1 neuron, softmax

Question 7.3 (Fill in the Blank)

Softmax outputs form a valid probability distribution because each output lies between 0 and 1 and all outputs together sum to _____.

Question 7.4 (Calculation)

If softmax produces $p = (0.2, 0.5, 0.3)$ and the true class is the second one, what is the value of $-\log(p_{\text{true}})$? Use $\log(0.5) = -0.693$.

- A) 0.693
- B) 1.609
- C) 1.204
- D) 0.500

8 Comprehensive Mixed Questions

Question 8.1 (Calculation)

A 2-2-1 MLP has sigmoid activation everywhere. Input $x_1 = 1$, $x_2 = 0$. Hidden layer weights: $w_{11} = 0.5$, $w_{12} = 0.3$, $b_1 = 0.2$; $w_{21} = 0.4$, $w_{22} = 0.6$, $b_2 = -0.1$. Output weights: $v_1 = 0.7$, $v_2 = -0.5$, $b_3 = 0.1$. Compute $z_1^{[1]}$.

- A) 0.700
- B) 0.500
- C) 0.300
- D) 0.900

Question 8.2 (Fill in the Blank)

For a layer with n_{in} inputs and n_{out} neurons, the number of parameters is _____ weights plus n_{out} biases.

Question 8.3 (Calculation)

A network has final scores $z = (0.5, -0.5)$. Using softmax, what is p_1 ? ($e^{0.5} = 1.6487$, $e^{-0.5} = 0.6065$)

- A) 0.5000
- B) 0.7311
- C) 0.2689
- D) 0.8808

Question 8.4 (MCQ)

Which of the following statements about the vanishing gradient problem is FALSE?

- A) It affects sigmoid and tanh activations
- B) It is caused by the activation derivative being small for large $|z|$
- C) It is completely solved by using ReLU in hidden layers
- D) It makes training deep networks difficult

Question 8.5 (Calculation)

Using gradient descent with $\alpha = 0.1$, $w = 2$, and gradient $\partial L / \partial w = 3$, compute w_{new} .

- A) 1.7
- B) 2.3
- C) 1.97
- D) 2.03

Question 8.6 (Fill in the Blank)

The maximum value of the sigmoid derivative $\sigma'(z)$ is _____.

Question 8.7 (MCQ)

In the NAG update, the look-ahead point is $w_{\text{look}} = w_{t-1} - \alpha\beta v_{t-1}$. Why $\alpha\beta$ and not just α ?

- A) It is an arbitrary convention
- B) $\alpha\beta v_{t-1}$ is exactly the already-committed part of the next step
- C) β makes it smaller and safer
- D) β is the learning rate for the look-ahead

Question 8.8 (Calculation)

Momentum with $\beta = 0.9$, $\alpha = 0.1$. $v_0 = 0$, gradients $g_1 = 4$, $g_2 = 4$, $g_3 = -1$. Compute v_3 .

Alph*) 0.5840

Alph*) 0.7600

Alph*) 0.4000

Alph*) 0.6840

Question 8.9 (Fill in the Blank)

The Adam optimizer combines the ideas of Momentum and _____.

Question 8.10 (MCQ)

Which of the following is NOT a valid reason to use mini-batch gradient descent over full-batch?

- A) Mini-batch uses less memory than full-batch
- B) Mini-batch performs more updates per epoch
- C) Mini-batch is faster per epoch than full-batch on GPU
- D) Mini-batch introduces beneficial gradient noise

9 Extended Calculations**Question 9.1 (Long Calculation)**

For a 1-1-1 network with sigmoid activation and squared error: $x = 1$, $w_1 = 0.5$ (input to hidden), $b_1 = 0$, $w_2 = 0.5$ (hidden to output), $b_2 = 0$, $y = 1$, $\alpha = 0.1$.

- (a) Compute the forward pass to find $\hat{y}$.
- (b) Compute δ_{output} .
- (c) Compute δ_{hidden} .
- (d) Compute the gradient $\partial L / \partial w_2$.
- (e) Update w_2 .

Question 9.2 (Long Calculation)

A 2-2-1 network with ReLU in hidden layer and sigmoid in output. $x = (1, 2)$, $y = 1$, $\alpha = 0.1$. Hidden weights: $w_{11} = 0.5$, $w_{12} = 0.3$, $b_1 = 0$; $w_{21} = -0.2$, $w_{22} = 0.6$, $b_2 = 0.1$. Output weights: $v_1 = 0.7$, $v_2 = -0.4$, $b_3 = 0$.

- (a) Compute hidden pre-activations $z_1^{[1]}, z_2^{[1]}$.
- (b) Apply ReLU to get $a_1^{[1]}, a_2^{[1]}$.
- (c) Compute output pre-activation $z_1^{[2]}$.
- (d) Apply sigmoid to get $\hat{y}$.
- (e) Compute the output error and δ_{output} .
- (f) Compute gradient for v_1 and update it.

10 Formula Sheet

Quantity	Formula
Sigmoid	$\sigma(z) = 1/(1 + e^{-z})$
Sigmoid derivative	$\sigma'(z) = \sigma(z)(1 - \sigma(z))$
Tanh derivative	$\tanh'(z) = 1 - \tanh^2(z)$
ReLU	$f(z) = \max(0, z)$
ReLU derivative	$f'(z) = 1$ if $z > 0$, else 0
MSE loss	$L = \frac{1}{2}(\hat{y} - y)^2$
Gradient descent	$w \leftarrow w - \alpha \frac{\partial L}{\partial w}$
Chain rule (output)	$\frac{\partial L}{\partial w} = (\hat{y} - y)\sigma'(z)x$
Momentum	$v_t = \beta v_{t-1} + (1 - \beta)g_t$
NAG look-ahead	$w_{\text{look}} = w - \alpha \beta v$
AdaGrad	$r_t = r_{t-1} + g_t^2$
RMSProp	$s_t = \rho s_{t-1} + (1 - \rho)g_t^2$
Adam first moment	$m_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t$
Adam second moment	$v_t = \beta_2 v_{t-1} + (1 - \beta_2)g_t^2$
Adam bias correction	$\hat{m}_t = m_t / (1 - \beta_1^t)$
Softmax	$p_i = e^{z_i} / \sum_j e^{z_j}$
Parameter count	$n_{\text{in}} \times n_{\text{out}} + n_{\text{out}}$

— End of Revision Guide —

Work all calculations by hand with a calculator.

Understand the *why*, not just the *how*.